Distributionally Robust LMI Control

Experimental SetUp

Giuseppe Fiengo

November 26, 2025

Contents

1 Experimental Setup **2**
1.1 Overview . . . 2
1.2 Plant, Ambiguity Set and Synthesis Pipeline . . . 2
1.3 Dataset Generation and Identification . . . 3
1.4 Closed–Loop Simulation . . . 4
1.5 Evaluation Metrics . . . 4
1.6 Comparison Protocol . . . 4
1.7 Implementation and Hardware . . . 5

2 Others **8**
2.1 Metrics . . . 9
2.2 Non-convex . . . 10
2.3 Solution Space . . . 11

Chapter 1

Experimental Setup

1.1 Overview

The numerical experiments assess the performance of a single distributionally robust output–feedback synthesis pipeline, referred to as DRO_LMI. The same optimization problem and implementation are used in two configurations:

- a *model–based* configuration, where the pipeline is initialized with the nominal state–space model and nominal disturbance description (`FROM_DATA = False`);
- a *data–driven* configuration, where the pipeline is initialized from identified models and disturbance statistics computed from simulated trajectories (`FROM_DATA = True`).

When the `ALL` flag is enabled, both configurations are evaluated over multiple independent runs with different random seeds, and statistics are reported across this ensemble.

1.2 Plant, Ambiguity Set and Synthesis Pipeline

The plant is a discrete–time linear time–invariant system

$$x_{k+1} = Ax_k + B_u u_k + B_w w_k, \qquad z_k = C_z x_k + D_{zu} u_k + D_{zw} w_k, \qquad y_k = C_y x_k + D_{yw} w_k,$$

with state $x_k \in \mathbb{R}^{n_x}$, input $u_k \in \mathbb{R}^{n_u}$, disturbance $w_k \in \mathbb{R}^{n_w}$, performance output $z_k \in \mathbb{R}^{n_z}$ and measured output $y_k \in \mathbb{R}^{n_y}$. The disturbance sequence is modeled as lying in a 2–Wasserstein ball of radius $\gamma > 0$ around a nominal zero–mean distribution with covariance Σ_{nom}. The ambiguity parameters $(\gamma, \Sigma_{\text{nom}})$ are fixed and shared by all experiments.

In both configurations, the controller is obtained by solving the same DRO_LMI synthesis problem, which produces a dynamic output–feedback law of the form

$$x^c_{k+1} = A_c x^c_k + B_c y_k, \qquad u_k = C_c x^c_k + D_c y_k.$$

The distinction between model–based and data–driven configurations lies only in the way the plant and disturbance quantities fed to the pipeline are obtained, as described below.

1.3 Dataset Generation and Identification

For the data–driven configuration, we generate open–loop datasets using a dedicated simulator. Each dataset consists of trajectories

$$\{x_k, u_k, y_k, z_k, x_{k+1}\}_{k=0}^{T-1},$$

where:

- the initial state x_0 is randomly drawn from a prescribed distribution (e.g. Gaussian or bounded box);
- the input sequence u_k is persistently exciting, typically multisine or pseudo–random binary sequences with fixed amplitude and frequency content;
- the disturbance w_k is sampled from the nominal distribution consistent with Σ_{nom}.

To improve robustness of the identification, we generate N_{sims} independent datasets with identical length T and different seeds. Instead of averaging trajectories (which would distort the dynamics), we compute a *representative* dataset by selecting the medoid of the collection: namely, the dataset whose stacked signals $(X, U, Y, Z, X_{\text{next}})$ minimize the total (weighted) squared distance to all other datasets in Frobenius norm. This medoid dataset is then used for all subsequent estimation steps.

From the medoid dataset we form the usual regression matrices

$$X_- = \begin{bmatrix} x_0 & \dots & x_{T-1} \end{bmatrix}, \quad U_- = \begin{bmatrix} u_0 & \dots & u_{T-1} \end{bmatrix}, \quad X_+ = \begin{bmatrix} x_1 & \dots & x_T \end{bmatrix},$$

and estimate the state and input matrices by least squares:

$$\begin{bmatrix} \hat{A} & \hat{B}_u \end{bmatrix} = X_+ \begin{bmatrix} X_- \\ U_- \end{bmatrix}^{\dagger}.$$

The corresponding residual $R_x = X_+ - \hat{A}X_- - \hat{B}_u U_-$ is interpreted as the contribution of the disturbance channel and used to construct an effective disturbance matrix $\hat{B}_w$ and an empirical covariance $\widehat{\Sigma}_w$. Analogous regressions are performed to obtain $(\hat{C}_y, \hat{D}_{yw})$ and $(\hat{C}_z, \hat{D}_{zu}, \hat{D}_{zw})$.

- In the *model–based* configuration (`FROM_DATA = False`), the pipeline uses the nominal matrices $(A, B_u, B_w, C_y, D_{yw}, C_z, D_{zu}, D_{zw})$ and the nominal disturbance description $(\gamma, \Sigma_{\text{nom}})$ directly, without any identification.
- In the *data–driven* configuration (`FROM_DATA = True`), the same pipeline is executed with the estimated quantities $(\hat{A}, \hat{B}_u, \hat{B}_w, \hat{C}_y, \hat{D}_{yw}, \hat{C}_z, \hat{D}_{zu}, \hat{D}_{zw})$ and an empirical disturbance covariance used to define the ambiguity set.

No other change is made to the optimization problem: solver settings, regularization terms and ambiguity parameters are kept identical between the two configurations.

1.4 Closed–Loop Simulation

Given a synthesized controller, we construct the interconnected closed–loop system with augmented state $\xi_k = (x_k, x_k^c)$. A dedicated simulator propagates the closed–loop dynamics over a finite horizon T_{sim}, using the same sampling time as in the identification phase. For each run we specify:

- an initial condition ξ_0, selected either as the origin, a random state, or a prescribed worst–case direction;
- a disturbance sequence w_k drawn from a nominal covariance consistent with the ambiguity set;
- optional reference signals, when present in the problem formulation.

The simulator records the trajectories of $(x_k, x_k^c, y_k, u_k, z_k)$ for post–processing.

1.5 Evaluation Metrics

To compare the model–based and data–driven configurations, we extract a set of scalar performance indicators from each closed–loop run. Typical metrics include:

- a finite–horizon quadratic cost J, combining performance output, state deviation and control effort over the trajectory;
- the spectral radius $\rho(A_{\text{cl}})$ of the closed–loop state matrix, used as a stability and robustness proxy;
- peak values of the performance output and control input (e.g. $\max_k \|z_k\|$, $\max_k \|u_k\|$);
- tail statistics (e.g. standard deviation of z_k and y_k over the last portion of the horizon) to quantify steady–state behavior;
- computational indicators such as solver objective, number of attempts, and wall–clock time required to solve the DRO_LMI problem.

When the `ALL` mode is enabled, each configuration is evaluated over multiple independent runs, each with its own random seeds for initial conditions and disturbances. For each metric we then report the empirical mean and standard deviation across runs, and we store per–run values for reproducibility and further analysis.

1.6 Comparison Protocol

The comparison between the two configurations is intentionally strict: for a given experimental setup (plant, ambiguity parameters, horizon, and solver settings) we:

1. synthesize a controller with the model–based configuration (`FROM_DATA = False`);
2. synthesize a controller with the data–driven configuration (`FROM_DATA = True`), using the medoid dataset and the associated estimates;

3. for each run in `ALL` mode, simulate both controllers under the same realizations of initial conditions and disturbance sequences;

4. compute all metrics for both configurations and aggregate results across runs.

This protocol isolates the impact of replacing the nominal model with its data–driven counterpart within the same DRO_LMI framework, since every other design choice is kept fixed.

1.7 Implementation and Hardware

All experiments are implemented in Python, using standard numerical and optimization libraries for matrix computations, convex optimization and data handling. The DRO_LMI synthesis problems are solved by off–the–shelf conic solvers, while simulations rely on custom routines for the plant, controller and closed–loop models. The experiments are run on a single workstation with multi–core CPU and sufficient RAM to handle the moderate state dimensions and trajectory lengths considered; no specialized hardware is required.

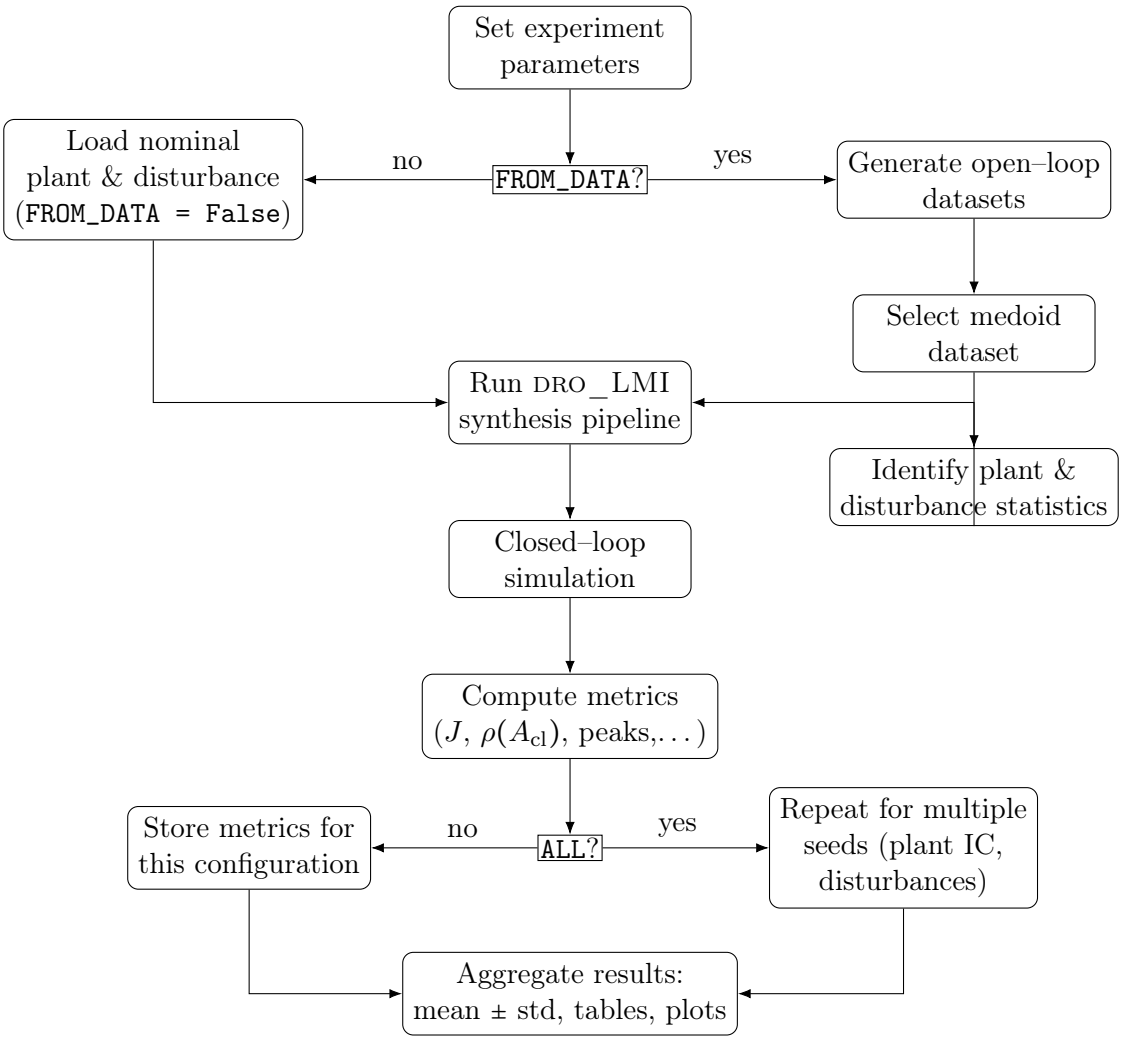


Figure 1.1: Overall experimental flow. The same DRO_LMI pipeline is run in model–based mode (`FROM_DATA = False`) and data–driven mode (`FROM_DATA = True`). The `ALL` flag controls whether metrics are computed for a single run or aggregated over multiple seeds.

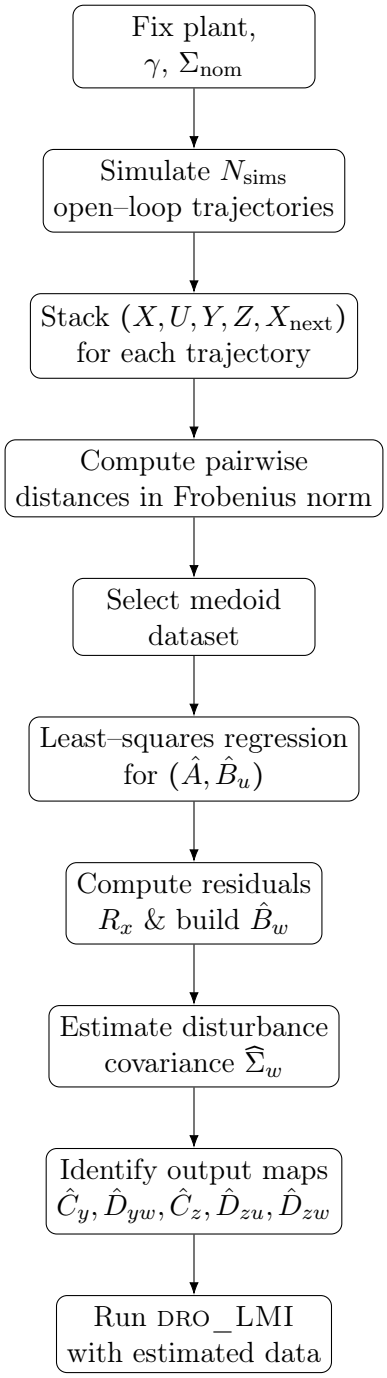


Figure 1.2: Data–driven branch of the experiment: generation of multiple open–loop trajectories, medoid selection, regression–based identification of the plant and disturbance statistics, and subsequent DRO_LMI synthesis with `FROM_DATA = True`.

Chapter 2

Others

Choice of Initial Conditions

The closed–loop simulations are carried out on the augmented state $\xi_k = (x_k, x_k^c)$, where x_k is the plant state and x_k^c is the controller state. Instead of drawing the initial condition at random, we construct x_0 and x_0^c so as to excite the most critical dynamical modes of the plant and the controller.

Let $A \in \mathbb{R}^{n_x \times n_x}$ denote the plant state matrix and $A_c \in \mathbb{R}^{n_{x^c} \times n_{x^c}}$ the controller state matrix obtained from the DRO_LMI synthesis. We compute their eigendecompositions

$$AV = V\Lambda, \qquad A_c V_c = V_c \Lambda_c,$$

and select, in each case, the eigenvalue with the largest real part (i.e. the least damped mode),

$$\lambda_* = \arg\max_{\lambda \in \sigma(A)} \Re(\lambda), \qquad \lambda_*^{(c)} = \arg\max_{\lambda \in \sigma(A_c)} \Re(\lambda),$$

with corresponding right eigenvectors $v_* \in \mathbb{C}^{n_x}$, $v_*^{(c)} \in \mathbb{C}^{n_{x^c}}$. Since the model is real, eigenvalues come in complex–conjugate pairs; we build real mode shapes by taking the real part and normalizing,

$$\tilde{v}_* := \frac{\Re(v_*)}{\|\Re(v_*)\|_2}, \qquad \tilde{v}_*^{(c)} := \frac{\Re(v_*^{(c)})}{\|\Re(v_*^{(c)})\|_2}.$$

The initial plant and controller states are then defined as

$$x_0 = r_x \tilde{v}_*, \qquad x_0^c = r_c \tilde{v}_*^{(c)},$$

with small scalar amplitudes $r_x, r_c > 0$ chosen so that the resulting deviations remain in the regime where the linear model is valid (e.g. $\|x_0\|_2, \|x_0^c\|_2$ on the order of a few percent of the nominal state magnitudes). The augmented initial condition is $\xi_0 = (x_0, x_0^c)$.

This eigenvector–based choice has two advantages compared to drawing x_0 and x_0^c as random Gaussian vectors:

- it guarantees that the initial condition is aligned with the least damped (i.e. most critical) modes of the plant and controller, which makes the transient a meaningful test of closed–loop damping and robustness;

- it removes the variability induced by arbitrary random initial conditions: all controllers are evaluated starting from the same structured perturbation, so differences in performance cannot be attributed to a "lucky" or "unlucky" random draw.

Random initial conditions are still useful for Monte–Carlo stress testing, but the eigenvector–based initialization provides a reproducible and deliberately challenging operating point for the comparisons presented here.
Given a state matrix $M \in \mathbb{R}^{n\times n}$:

1. Compute eigenvalues and eigenvectors:

$$MV = V\Lambda.$$

2. Select the eigenvalue with largest real part:

$$\lambda_* = \arg\max_{\lambda_i \in \sigma(M)} \Re(\lambda_i).$$

3. Extract the corresponding eigenvector v_*.
4. Build a real mode shape:

$$\tilde{v} = \frac{\Re(v_*)}{\|\Re(v_*)\|_2}.$$

5. Scale it to obtain the initial condition:

$$x_0 = r\,\tilde{v}, \qquad r \text{ small (e.g., } r = 0.05).$$

2.1 Metrics

Category	Metric	Description
Performance	J	Finite–horizon quadratic cost on (z_k, x_k, u_k) over the simulation horizon.
Stability	$\rho(A_{\mathrm{cl}})$	Spectral radius of the closed–loop state matrix.
Performance	$z_{\max}$	Peak performance output, $z_{\max} = \max_k \|z_k\|_2$.
Performance	$u_{\max}$	Peak control effort, $u_{\max} = \max_k \|u_k\|_2$.
Steady–state	σ_z^{tail}	Standard deviation of z_k over the last portion of the horizon.
Steady–state	σ_y^{tail}	Standard deviation of y_k over the last portion of the horizon.
Computational	obj	Optimal objective value of the DRO_LMI synthesis problem.
Computational	t_{solve}	Wall–clock time required by the conic solver to compute the controller.
Computational	attempts	Number of solver attempts / restarts until a feasible solution is obtained.
Diagnostic	stress	Scalar diagnostic summarizing how "tight" or conservative the design is.
Aggregation	$\mu \pm \sigma$	Mean and standard deviation of each metric over all runs in `ALL` mode.

Table 2.1: Closed–loop performance and computational metrics used for comparison.

2.2 Non-convex

Algorithm 1 Nonconvex data–driven DRO–LMI (Young–Schur–free)

Require: simulator/API, $(\gamma, \Sigma_{\text{nom}})$, model type, N_{sims}, ρ
Ensure: (K, L, M, N), $(A_{\text{true}}, B, C, D, P)$

1. Data / medoid

if open–loop mode **then**
  Simulate N_{sims} datasets $(X^{(i)}, U^{(i)}, Y^{(i)}, Z^{(i)}, X^{(i)}_{\text{next}})$.
  Align to common length $T_{\min}$; stack signals for each i.
  Select medoid index $i^\star$ via Frobenius distances.
  Set $(X_-, U_-, Y_-, Z_-, X_+) \leftarrow (X^{(i^\star)}, U^{(i^\star)}, Y^{(i^\star)}, Z^{(i^\star)}, X^{(i^\star)}_{\text{next}})$.
else
  Obtain $(X_-, U_-, Y_-, Z_-, X_+)$ from API.

2. Identification & bounds

$D_x \leftarrow \begin{bmatrix} X_- \\ U_- \end{bmatrix}, \quad [\hat{A}\ \hat{B}_u] \leftarrow X_+ D_x^\dagger.$

$R \leftarrow X_+ - \hat{A}X_- - \hat{B}_u U_-.$

From R: estimate B_w, n_w, disturbance sequence $W_= B_w^\dagger R$.

From $(X_-, U_-, W_, Y_-, Z_-)$: estimate (C_y, D_{yw}), (C_z, D_{zu}, D_{zw}).

Optionally update $(\Sigma_{\text{nom}}, \gamma)$ and augment (B_w, D_{zw}, D_{yw}).

Compute $\beta = \|R\|_{\text{F}}/\sigma_{\min}(D_x)$, $\beta_A = \beta\sqrt{\frac{n_x}{n_x+n_u}}$, $\beta_B = \beta\sqrt{\frac{n_u}{n_x+n_u}}$.

3. Decision variables & lifted model

Variables: $X, Y \in \mathbb{S}^{n_x}$, $Q \in \mathbb{S}^{n_w}$, K, L, M, N, $\Delta A, \Delta B$, $\lambda \geq 0$.

$P \leftarrow \begin{bmatrix} Y & I \\ I & X \end{bmatrix}.$

Build $(A_{\text{nom}}, B, C, D)$ from $(\hat{A}, \hat{B}_u, B_w, C_y, D_{yw}, C_z, D_{zu}, D_{zw})$ and (K, L, M, N).

Build structured $E_A(\Delta A)$, $E_B(\Delta B)$, set $E = E_A + E_B, \quad A_{\text{true}} = A_{\text{nom}} + E$.

4. Constraints

if model = correlated **then**
  Form kernel $\Xi(A_{\text{true}}, B, C, D, P, Q, \lambda)$.
  Constraint: $\lambda_{\min}(\Xi) + \varepsilon \leq 0$.
else
  Form $\Xi_1(A_{\text{true}}, C, P)$, $\Xi_2(B, D, P, Q, \lambda)$.
  Constraints: $\lambda_{\min}(\Xi_1) + \varepsilon \leq 0$, $\lambda_{\min}(\Xi_2) + \varepsilon \leq 0$.

$\|\Delta A\|_{\text{F}} \leq \beta_A, \quad \|\Delta B\|_{\text{F}} \leq \beta_B.$

$\lambda_{\min}(P) \geq \varepsilon, \quad \lambda_{\min}(Q) \geq \varepsilon, \quad \lambda \geq 0.$

5. Objective

$f_{\text{DRO}} \leftarrow \text{trace}(Q\Sigma_{\text{nom}}) + \lambda\gamma^2.$

$f_{\text{reg}} \leftarrow \rho\big(\|K\|_{\text{F}}^2 + \|L\|_{\text{F}}^2 + \|M\|_{\text{F}}^2 + \|N\|_{\text{F}}^2 + \|X\|_{\text{F}}^2 + \|Y\|_{\text{F}}^2\big)$ (or 0).

$f \leftarrow f_{\text{DRO}} + f_{\text{reg}}.$

6. NLP solve & unpack

Stack all variables into x, constraints into $g(x)$.

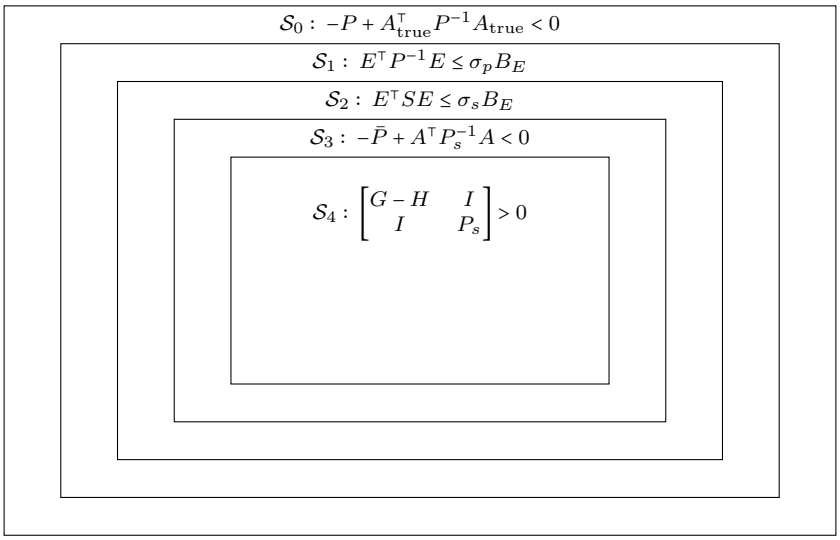
Solve $\min_x f(x)$ s.t. $g(x) \leq 0$ with IPOPT (box bounds on x, $x_0 = 0$).

Unpack $X^\star, Y^\star, Q^\star, K^\star, L^\star, M^\star, N^\star, \Delta A^\star, \Delta B^\star, \lambda^\star$.

Rebuild $P^\star, A^\star_{\text{nom}}, E^\star, A^\star_{\text{true}}, B^\star, C^\star, D^\star$.

return $(K^\star, L^\star, M^\star, N^\star)$, $(A^\star_{\text{true}}, B^\star, C^\star, D^\star, P^\star)$.

2.3 Solution Space



$$\mathcal{S}_4 \subset \mathcal{S}_3 \subset \mathcal{S}_2 \subset \mathcal{S}_1 \subset \mathcal{S}_0$$